

```
$ node breakfast.js --mode=async
```

JavaScript □□□

```
// 00000000 Promise 0 async/await
```

sync

□□□□

promise

□□□□□

await

□□□□□

event loop

□□□□

\$ cat menu.md



01 /



vs



02 / Promise

then/catch

03 / async & await

vs

04 / Event Loop

VIP

async

PART 01



□□□□□□□□□□

□□□□□□□□□

```
1  function makeCoffee() {
2      console.log("☕☕☕☕...");
3      const start = Date.now();
4      while (Date.now() - start < 2000) {} // ☐☐☐
5      console.log("☕☕☕☕");
6  }
7
8  function prepareBreakfastSync() {
9      makeCoffee(); // 2s
10     makeToast(); // 1.5s
11     fryEgg();     // 1s
12 }
```

CONSOLE — 4.5s

☕☕☕☕...

☐☐☐☐☐

☕☕☕☕...

☐☐☐☐☐☐

☐☐☐☐☐☐☐☐☐☐☐☐

```
CONSOLE - ~2s
```

0000...

0000...

0000...

0000000000000000

00000000000000

```

graph TD
    subgraph Row1
        B1_1[ ]
        B1_2[ ]
        B1_3[ ]
    end
    subgraph Row2
        B2_1[ ]
        B2_2[ ]
        B2_3[ ]
    end
    subgraph Row3
        B3_1[ ]
        B3_2[ ]
        B3_3[ ]
    end
    subgraph Row4
        B4_1[ ]
        B4_2[ ]
        B4_3[ ]
        B4_4[ ]
    end
    B1_1 --- B1_2 --- B1_3
    B2_1 --- B2_2 --- B2_3
    B3_1 --- B3_2 --- B3_3
    B4_1 --- B4_2 --- B4_3 --- B4_4
    style B1_1 fill:#fff,stroke:#fff
    style B1_2 fill:#fff,stroke:#fff
    style B1_3 fill:#fff,stroke:#fff
    style B2_1 fill:#fff,stroke:#fff
    style B2_2 fill:#fff,stroke:#fff
    style B2_3 fill:#fff,stroke:#fff
    style B3_1 fill:#fff,stroke:#fff
    style B3_2 fill:#fff,stroke:#fff
    style B3_3 fill:#fff,stroke:#fff
    style B4_1 fill:#fff,stroke:#fff
    style B4_2 fill:#fff,stroke:#fff
    style B4_3 fill:#fff,stroke:#fff
    style B4_4 fill:#fff,stroke:#fff
  
```

Diagram illustrating a sequence of data elements (boxes) and their corresponding indices (dots).

- Row 1: Four white boxes followed by three dots (indices 0, 1, 2, 3).
- Row 2: Four white boxes followed by three dots (indices 4, 5, 6, 7).
- Row 3: Ten green boxes (indices 8, 9, 10, 11, 12, 13, 14, 15, 16, 17).
- Row 4: Four white boxes followed by three yellow boxes (indices 18, 19, 20, 21, 22, 23, 24).

Diagram illustrating a 3D array structure with dimensions 3x3x3. The first two dimensions are represented by a 3x3 grid of green squares, and the third dimension is represented by a 3x3 grid of yellow squares.

A diagram of a 16-bit register. It consists of two rows of eight slots each. The top row of eight slots is green, and the bottom row of eight slots is yellow.

[illegible]



□ □ □ □ □ □ □ □ □ □

[illegible]

PART 02

Promise

□□□□□□□□□□

□□□□□□□□

pending

□□□□□□

fulfilled

□□□□resolve("🍌🍌")

rejected

□□□□reject("🍌🍌")

```
1 let myPromise = new Promise((resolve, reject) => {
2   let success = true;
3   setTimeout(() => {
4     if (success) resolve("🍌🍌");
5     else reject("🍌🍌");
6   }, 2000);
7 });
8
9 myPromise.then(result => console.log(result))
10 .catch(error => console.log(error));
```


□□□□□ Promise.race + AbortController

Promise.race



AbortController

PART 03

async / await

□□□□□□□□□□

then **NOGRAPH** → **await**

```
await then async/await then
```



```
1  async function serial() {  
2    const coffee = await makeCoffee();  
3    const toast = await makeToast();  
4    return [coffee, toast];  
5  }
```

TOTAL TIME

$\approx 2 + 1.5 = 3.5s$

□□

token API — □□□□□



```
1  async function parallel() {  
2    const [coffee, toast] = await Promise.all([  
3      makeCoffee(),  
4      makeToast(),  
5    ]);  
6    return [coffee, toast];  
7  }
```

TOTAL TIME

$\approx \max(2, 1.5) = 2s$

□□

token API

Promise.all vs allSettled

	Promise.all	Promise.allSettled
□□□□□	 reject □□□□	□□□□□□□
□□	□□□□	{status, value/reason} □□
□□	□□□□□□□	□□□□□□□□□□

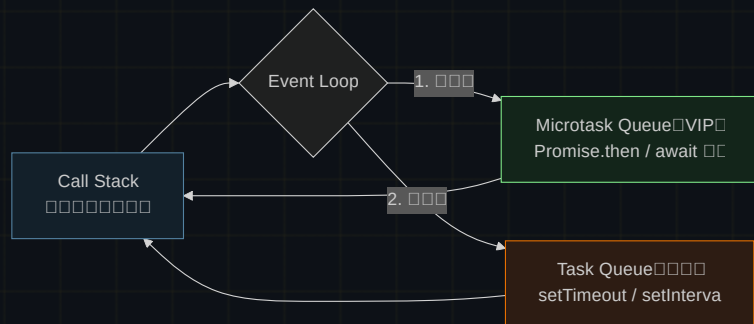
\$ npm run all □□□□□□□□allSettled □□□□□□□

PART 04

Event Loop

0000 VIP 0000000000

□ □ □ □ □ □ □ □ □



□□□□VIP□
□□□□□□□□□□

□□□□□□□□
□□□□□□□□□□□□



```
1 console.log("1");
2 setTimeout(() => console.log("2"), 0);
3 Promise.resolve().then(() => console.log("3"));
4 console.log("4");
```

ANSWER

1 → 4 → Call Stack

→ 3 → VIP

→ 2 → 0

1 → 4 → 3 → 2

□ □ □ □ □ □ □ □ □ □ □ □ □ □

□□□□□pending → fulfilled / rejected

await Promise.all

□ □ □ □ □ □ □ □ □ □ □ □



Breakfast served.

fetch AbortController

```
$ node breakfast.js --mode=async
```

promise → await → race → event loop